🌐 Section 8 — Container Networking

This section answers the key questions:

1. **Does a container automatically have network access?**
2. **Is inter-container communication common?**
3. **How does container networking work?**

8.1 Do Containers Automatically Have Network Access?

Short answer: Yes.

When a container starts, Docker automatically:

- Creates a **network namespace** for the container
- Connects it to a **virtual Ethernet pair (veth)**
- Attaches that veth to the host's **docker0 bridge**
- Gives the container:
  - a private IP (e.g., `172.17.0.2`)
  - outbound access to the internet via NAT

This means:

8.2 How Containers Reach the Outside World

Default Docker network: **bridge mode**.

The host runs a virtual bridge:

```
[container eth0] — veth — [docker0 bridge] — NAT — Internet
```

- Outbound traffic from the container is NAT'd through the host.
- Inbound traffic is blocked unless you use `-p` to publish ports.

Example:

```
docker run -p 8080:80 nginx
```

This maps:

- **host port 8080 → container port 80**

8.3 Do Images Define Networking?

No.
Images do not define networking.

Networking is defined at **runtime**:

- via Docker's default bridge
- or via custom networks
- or via host networking mode
- or via user-defined overlay networks (Docker Swarm / Kubernetes)

The Dockerfile has **no networking configuration** except environment variables that applications might read.

8.4 Inter-Container Communication (Yes, Very Common)

Inter-container networking is fundamental for:

- microservices
- parallel tasks
- pipeline-style architectures
- distributed data processing
- multi-service stacks (API + DB, etc.)

Containers on the same Docker network:

- get unique private IPs
- can reach each other by **name** (DNS-based service discovery)

8.5 Types of Docker Networking

1. Bridge Network (default)

- Containers receive internal IPs.
- Outbound internet works via NAT.
- Inbound requires port publishing.
- Containers on the same bridge can communicate.

2. Host Network (optional)

Container shares the host network namespace:

```
docker run --network host ...
```

- No isolation
- Highest performance

8.6 Example: API-Calling Container

Assume a Python container that needs to fetch data:

```
import requests
print(requests.get("https://api.github.com").status_code)
```

You don’t need any special Dockerfile networking config.

Running it:

```
docker run --rm my-python-app
```

Works immediately because:

- `eth0` exists inside the container
- The default route points to the docker bridge
- NAT handles outbound traffic

8.7 Example: Inter-Container Communication for Parallelism

Run three worker containers and one controller:

```
docker network create cluster

docker run -d --name worker1 --network cluster worker-image
docker run -d --name worker2 --network cluster worker-image
docker run -d --name worker3 --network cluster worker-image
docker run -it --name controller --network cluster controller-image
```

Inside the controller:

```
ping worker1
curl http://worker2:8000/task
```

This pattern is extremely common in:

8.8 OS Concepts Illustrated by Networking

Container networking relies on:

Network namespaces

- Isolated routing tables
- Isolated ARP tables
- Isolated interfaces (`eth0` , `lo`)

Virtual Ethernet Pairs (veth)

- Each container gets one end
- Host bridge gets the other

Bridges / NAT

- Kernel-level bridge acts like a virtual switch

8.9 Quick Summary

- Containers **automatically** get outbound network access.
- Inbound access requires **port publishing**.
- Images do **not** define networking behavior.
- Inter-container communication is **common and fundamental**.
- User-defined networks provide **DNS-based service discovery**.
- Container networking demonstrates:
  - network namespaces
  - veth pairs
  - bridge networking
  - NAT
  - kernel-level virtual switching